```
 ...

 val drainingControl: Consumer.DrainingControl[_] =
      Transactional                                                          ❶
        .source(
          consumerSettings,
          Subscriptions.topics("test5")
        )
        .map { msg: TransactionalMessage[String, String] =>                  ❷
          ProducerMessage.single(
            new ProducerRecord[String, String](
              "test6",
              msg.record.key,
              msg.record.value),
            msg.partitionOffset)                                             ❸
        }
        .toMat(Transactional.sink(producerSettings, transactionalId))(       ❹
          Consumer.DrainingControl.apply)
        .run()
}
```

❶ Transanctional.source instead of Consumer.committableSource

❷ TransactionalMessage instead of CommittableMessage

❸ partitionOffset from TransactionalMessage instead of committableOffset

❹ Sink with a transactionalId that committableSink doesn't have

NOTE The property `isolation.level` must be `read-committed`. So, you need to add this property and value to the `producerSettings`.

The `commit` settings are like auto-commit, which you saw in the `ConsumerSettings` with the `plainSink`. However, instead of `auto.commit.interval.ms`, for transactions, this is set in `akka.kafka.producer` via the `eos-commit-interval` property. The default value is 100 milliseconds.

The tradeoff between at-least-once and effectively-once is performance, as you may have guessed. This is because the stream